算法计量与验证实验报告

姓名：　学号：

运行环境说明

- **操作系统**：macOS
- **python 版本**：3.13.2
- **代码仓库**：https://github.com/maidou07/dsa2026.git

I：验证 list 的按索引取值为 O(1)

代码

```python
import timeit

# 在固定list大小为1000下不同点取索引
print("验证1：固定list大小 = 1000，在不同点取索引")
for i in [0, 499, 999]:
    setup = "lst = list(range(1000))"
    run = f"x = lst[{i}]"
    t = timeit.Timer(run, setup)
    total = t.timeit(number = 1000000)
    print(f"在 index = {i} 取值1000000次所用时间：{total:.5f}秒")

print("---")

# 改变list大小，在相同点取索引
print("验证2：改变list大小，在相同点取索引")
list_sizes = [1000, 10000, 100000]
for i in list_sizes:
    setup = f"lst = list(range({i}))"
    run = "x = lst[0]"
    t = timeit.Timer(run, setup)
    total = t.timeit(number = 1000000)
    print(f"list大小为{i}下：在 index = 0 处取值1000000次所用时间：{total:.5f}秒")
```

数据说明

选择两个方面进行完整详细验证:

- 一方面固定 `list` 的大小，在**不同索引位置**进行取值操作，选取 `list` 大小为 1000，在头、中、尾（0，499，999）取值
- 另一方面固定在 0 处取值，**改变 list 大小**（1000，10000，100000）

运行实验说明

代码通过 `timeit` 计时，并取值 1000000 次，输出总用时以减小单次调用的时间涨落。

实验结果分析

实验结果

```
验证1: 固定list大小 = 1000，在不同点取索引
在 index = 0 取值1000000次所用时间: 0.00492秒
在 index = 499 取值1000000次所用时间: 0.00495秒
在 index = 999 取值1000000次所用时间: 0.00488秒
---
验证2: 改变list大小，在相同点取索引
list大小为1000下: 在 index = 0 处取值1000000次所用时间: 0.00464秒
list大小为10000下: 在 index = 0 处取值1000000次所用时间: 0.00487秒
list大小为100000下: 在 index = 0 处取值1000000次所用时间: 0.00447秒
```

分析

可以看到，在不同大小的 `list` 的不同位置取值，所用的时间都基本相同，说明算法的运行时间不依赖于取值位置以及 `list` 的大小 N，故 `list` 按索引取值的操作时间复杂度为 $\mathrm{O}(1)$。

II: 验证 dict 的 get item 和 set item 都是 O(1) 的

代码

```python
import timeit
# 1. 在固定dict大小 = 1000下在不同key取item
print("验证1：固定dict大小 = 1000，在不同key执行get item")
for key in [0, 499, 999]:
    setup = "dic = {k:k for k in range(1000)}"
    run = f"x = dic[{key}]"
    t = timeit.Timer(run, setup)
    total = t.timeit(number=1000000)
    print(f"在 key = {key} 取数1000000次所用时间：{total:.5f}秒")
print("---")
# 2. 改变dict大小，在相同key取item
print("验证2：改变dict大小，在相同key执行get item")
dict_sizes = [1000, 10000, 100000]
for i in dict_sizes:
    setup = f"dic = {{k:k for k in range({i})}}"
    run = "x = dic[0]"
    t = timeit.Timer(run, setup)
    total = t.timeit(number=1000000)
    print(f"dict大小为{i}下：在 key = 0 处取数1000000次所用时间：{total:.5f}秒")
print("---")
# 3. 在固定 dict大小 = 1000 下在不同key执行set item
print("验证3：固定dict大小 = 1000，在不同key执行set item")
for key in [0, 499, 999]:
    setup = "dic = {k:k for k in range(1000)}"
    run = f"dic[{key}] = 0"
    t = timeit.Timer(run, setup)
    total = t.timeit(number=1000000)
    print(f"在 key = {key} 修改1000000次所用时间：{total:.5f}秒")
print("---")
# 4. 改变dict大小，在相同key 执行 set item
print("验证4：改变dict大小，在相同key执行set item")
dict_sizes = [1000, 10000, 100000]
for i in dict_sizes:
    setup = f"dic = {{k:k for k in range({i})}}"
    run = "dic[0] = 0"
    t = timeit.Timer(run, setup)
    total = t.timeit(number=1000000)
    print(f"dict大小为{i}下：在 key = 0 处修改1000000次所用时间：{total:.5f}秒")
```

数据说明

我们每次将 `dict` 初始化为 `{key=k: value=k}`. 然后使用和 I 相同的方式，分别对 `get item` 和 `set item` 都进行了两方面的测试：

- **不同键位置的测试**：固定 `dict` 大小为 1000，分别在 key 为 0、499、999 处进行读写操作。
- **改变 dict 大小的同键测试**：设置 `dict` 大小分别为 1000、10000、100000，统一对 `key` 为 0 的位置进行读写操作。

运行实验说明

借助 `timeit.Timer` 模块，对 `dic[key]` (读取) 和 `dic[key] = 0` (赋值) 这两条核心语句分别循环执行 $1,000,000$ 次，记录总耗时以减小单次调用的时间涨落。

实验结果分析

实验结果

```
验证1: 固定dict大小 = 1000，在不同key执行get item
在 key = 0 取数1000000次所用时间: 0.00637秒
在 key = 499 取数1000000次所用时间: 0.01106秒
在 key = 999 取数1000000次所用时间: 0.01096秒
---
验证2: 改变dict大小，在相同key执行get item
dict大小为1000下: 在 key = 0 处取数1000000次所用时间: 0.00624秒
dict大小为10000下: 在 key = 0 处取数1000000次所用时间: 0.00643秒
dict大小为100000下: 在 key = 0 处取数1000000次所用时间: 0.00647秒
---
验证3: 固定dict大小 = 1000，在不同key执行set item
在 key = 0 修改1000000次所用时间: 0.01007秒
在 key = 499 修改1000000次所用时间: 0.01288秒
在 key = 999 修改1000000次所用时间: 0.01229秒
---
验证4: 改变dict大小，在相同key执行set item
dict大小为1000下: 在 key = 0 处修改1000000次所用时间: 0.00941秒
dict大小为10000下: 在 key = 0 处修改1000000次所用时间: 0.00927秒
dict大小为100000下: 在 key = 0 处修改1000000次所用时间: 0.00969秒
```

分析

1. 当数据量级发生改变时，操作时间仍基本相同。这证明了基于 Hash 表实现的字典能够以 $O(1)$ 复杂度完成寻址和修改操作。

2. **关于 `key=0` 耗时异常偏低的讨论**：由于 Python 底层优化——Python 的**小整数缓存池**机制，程序预先缓存了 $[-5, 256]$ 的小整数对象，导致 `key=0` 耗时异常偏低。
但是可以看到其余 `key` 耗时基本相同，说明算法总体趋势依然严格遵循 $O(1)$ 的时间复杂度模型。

III: 比较 list 和 dict 的 del 操作时间复杂度

代码

```python
import timeit
import matplotlib.pyplot as plt
import statistics
import os
current_dir = os.path.dirname(os.path.abspath(__file__))
fig_path = os.path.join(current_dir, '..', 'img')
lst_times = []
dct_times = []

def test(N = 50000):
    global lst_times
    global dct_times

    for i in range(0, N, 1000):  # 枚举del操作的位置索引(list:index, dict:key)，间距为100

        # list 计时部分
        setup1 = f"lst = list(range({N}))"
        run1 = f"del(lst[{i}])"
        lst_time = timeit.Timer(run1, setup1).repeat(repeat = 100, number = 1)
        lst_times.append(statistics.mean(lst_time))

        # dict 计时部分
        setup2 = f"dct = {{k:k for k in range({N})}}"
        run2 = f"del dct[{i}]"
        dct_time = timeit.Timer(run2, setup2).repeat(repeat = 100, number = 1)
        dct_times.append(statistics.mean(dct_time))
test()
# plt绘图
plt.figure(figsize=(10, 6))
plt.plot(range(0, 50000, 1000), lst_times, label='List del x[i]', color='blue', marker='o', markersize=2)
plt.plot(range(0, 50000, 1000), dct_times, label='Dict del d[i]', color='red', linestyle='--')

plt.title('Comparison of del Operation: List vs Dictionary')
plt.xlabel('Index/Key to Delete (i)')
plt.ylabel('Avg Time for 100 Repetitions (seconds)')
plt.legend()
plt.grid(True)
plt.savefig(os.path.join(fig_path, 'del_comparison.png'))
plt.show()
```

数据说明

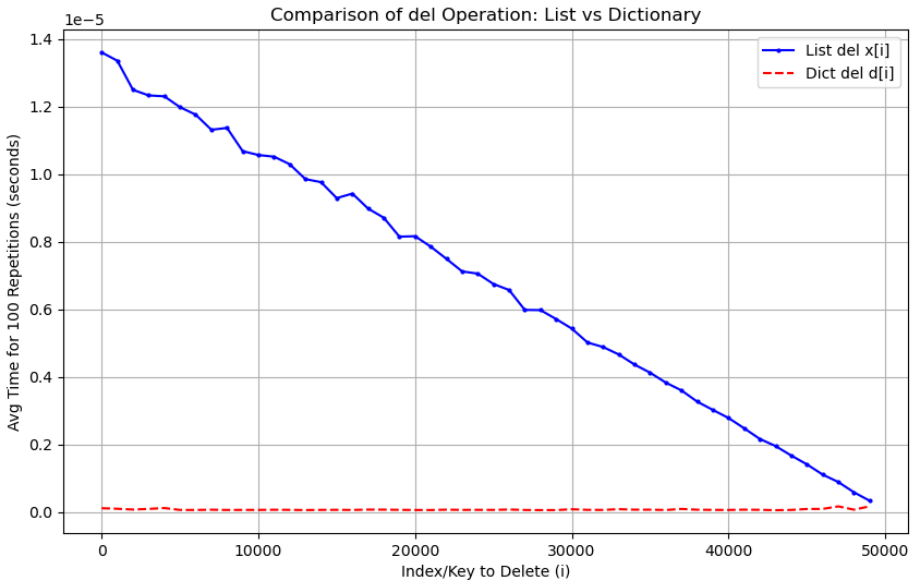
预先构造包含 50000 个元素的 list 和 dict。在 0 到 50000 的索引（或键）范围内，以 1000 为步长，分别枚举并删除对应位置的元素。

运行实验说明

考虑到 `del` 操作的不可逆性，实验在 `setup` 阶段每次都重新生成完整的测试结构。对于每个选定的删除位置 i，重复执行 100 次 `del` 并利用 `statistics.mean()` 求取平均耗时。最后通过 `matplotlib` 将得到的数据点绘制成对比折线图。

实验结果分析

实验结果:



分析：从绘制的耗时折线图可以直观地得出以下结论：

- **对于 dict（红色虚线）**：`del` 操作的耗时几乎贴近 X 轴并保持水平，完全不随被删除元素所处位置的变化而改变。这印证了字典的 `del` 也是通过哈希定位，时间复杂度为 $O(1)$。
- **对于 list（蓝色实线）**：耗时呈现出明显的**线性下降趋势**。这是由于 Python 的 `list` 在底层由连续内存的数组实现，当删除索引为 k 的元素时，必须将其后方 $N-1-k$ 个元素整体向前移动一位。因此，当 k 越小（越靠近列表头部），需要移动的元素越多，耗时最长，整体呈现 $O(N)$ 的复杂度特性；当 k 靠近尾部时，无需移动其他元素，耗时逐渐逼近 $O(1)$，故 `list` 的 `del` 操作可以总结为 $O(N-k)$ 的时间复杂度。

IV: 验证 `list.sort()` 的时间复杂度为 $O(N \log N)$

代码

```python
import timeit
import statistics, math

# 枚举数据大小
sizes = [10000, 20000, 40000, 80000, 160000]

def test():
    # 循环计算sort时间
    for i in sizes:
        setup = f"import random; lst = [random.randrange(10*6) for _ in range({i})]"
        run = "lst.sort()"
        t = timeit.Timer(run, setup)
        sort_time = statistics.mean(t.repeat(repeat = 1000, number = 1))   #取1000次平均 减小涨落影响
        ratio = sort_time / (i * math.log(i))   # 计算time/nlogn 证明O(nlogn)

        print(f"{i} : {10**3*sort_time:.3f}ms {10**9*ratio:.3f}*10^(-9)")

test()
```

数据说明

枚举不同的数据规模：$10000, 20000, 40000, 80000, 160000$。针对每种规模，利用 `random` 模块生成完全无序的随机整数列表。

运行实验说明

由于 `sort()` 算法在排序之后，再次排序的时间复杂度会退化，因此在 `timeit` 的 `setup` 中必须每次都重新生成无序数组。实验对每种规模重复执行 1000 次求得平均时间 t。

为了验证其是否符合 $O(N \log N)$ 的规律，我们计算了比值 $ratio = \frac{t}{N \log N}$。如果时间复杂度符合该模型，则比值应趋近于一个常数。

实验结果分析

实验结果

输出格式为:
数据规模: 用时 (ms) ratio

```
10000 : 0.446ms 4.839*10^(-9)
20000 : 0.898ms 4.536*10^(-9)
40000 : 1.814ms 4.280*10^(-9)
80000 : 3.667ms 4.061*10^(-9)
160000 : 7.465ms 3.893*10^(-9)
```

分析

通过代码输出结果可以观察到：尽管数据规模 N 成倍增加，但计算出的比值却基本相同。说明用时 t 与 $N \log N$ 存在严格比例关系，验证了 Python 内置的 `sort()` 在处理一般无序数据时，时间复杂度满足 $O(N \log N)$ 模型。